```
import { q, one } from '../db.js';
import { getPackage } from './registry.js';
import { grant, revokeForInstall } from './policy.js';
import { emit } from './events.js';

const TYPES = {
  text: 'text', string: 'text', longtext: 'text',
  int: 'integer', integer: 'integer', bigint: 'bigint',
  number: 'numeric', money: 'numeric(12,2)',
  bool: 'boolean', boolean: 'boolean',
  date: 'date', time: 'time', timestamp: 'timestamptz',
  json: 'jsonb', uuid: 'uuid',
};

// Installing an app is what makes it real for one business:
// its declared tables get provisioned, its capabilities get granted,
// and its public sections get registered. It never gets DDL rights itself.
export async function install({ businessId, packageKey, settings = {}, grantTo =
  const pkg = getPackage(packageKey);
  if (!pkg) throw new Error(`no such package: ${packageKey}`);
  const m = pkg.manifest;

  const existing = await one(
    `select * from install where business_id = $1 and package_key = $2`, [busines
  );
  if (existing && existing.status === 'active') return { install: existing, alrea

  for (const dep of m.requires ?? []) {
    const has = await one(
      `select 1 from install where business_id = $1 and package_key = $2 and stat
      [businessId, dep]
    );
    if (!has) throw new Error(`${packageKey} requires ${dep}, which is not instal
  }

  const row = existing
    ? await one(`update install set status='active', version=$1, settings=$2, rem
                 where id=$3 returning *`, [m.version, JSON.stringify(settings),
    : await one(
        `insert into install (business_id, package_id, package_key, version, sett
         select $1, p.id, $2, $3, $4::jsonb from package p where p.key = $2 and p
         returning *`,
        [businessId, packageKey, m.version, JSON.stringify(settings)]
      );
```

```
  for (const [logicalName, def] of Object.entries(m.schema ?? {})) {
    await provision({ install: row, businessId, packageKey, logicalName, def });
  }

  for (const capKey of m.capabilities ?? []) {
    for (const role of grantTo) {
      await grant({ businessId, role, capability: capKey, disposition: dispositio
    }
  }

  for (const section of m.public ?? []) {
    await one(
      `insert into projection_map (business_id, install_id, section_key, title, i
       values ($1,$2,$3,$4,$5,$6,$7)
       on conflict (business_id, section_key) do update set title = excluded.titl
         renderer = excluded.renderer, visible = true
       returning *`,
      [businessId, row.id, section.key, section.title, section.icon ?? null,
       section.order ?? 100, section.renderer]
    );
  }

  if (pkg.module.onInstall) {
    await pkg.module.onInstall({ businessId, install: row, settings });
  }

  await emit({ businessId, topic: 'package.installed', payload: { packageKey, ver
  return { install: row };
}

function dispositionFor(manifest, capKey) {
  const sensitive = manifest.sensitive ?? [];
  return sensitive.includes(capKey) ? 'ask' : 'auto';
}

// JSON field declarations become a real table. The kernel adds id and
// business_id itself so a package cannot forget tenancy or lie about it.
async function provision({ install, businessId, packageKey, logicalName, def }) {
  const physical = `pkg_${packageKey.replace(/-/g, '_')}__${logicalName}`;

  const existing = await one(
    `select * from provisioned_table where physical_name = $1`, [physical]
  );
  if (existing) {
    await q(`update provisioned_table set install_id = $1 where id = $2`, [instal
    return existing;
```

```
  }

  const cols = [
    `id uuid primary key default gen_random_uuid()`,
    `business_id uuid not null references business(id) on delete cascade`,
  ];

  for (const [field, spec] of Object.entries(def.fields)) {
    if (!/^[a-z0-9_]+$/.test(field)) throw new Error(`bad field name: ${field}`);
    const t = typeof spec === 'string' ? { type: spec } : spec;
    const sqlType = TYPES[t.type];
    if (!sqlType) throw new Error(`unknown field type "${t.type}" on ${logicalNam
    let col = `"${field}" ${sqlType}`;
    if (t.required) col += ' not null';
    if (t.default !== undefined) col += ` default ${literal(t.default)}`;
    cols.push(col);
  }
  cols.push(`created_at timestamptz not null default now()`);

  await q(`create table "${physical}" (${cols.join(', ')})`);
  await q(`create index on "${physical}" (business_id)`);

  return one(
    `insert into provisioned_table (install_id, package_key, logical_name, physic
     values ($1,$2,$3,$4,$5) returning *`,
    [install.id, packageKey, logicalName, physical, JSON.stringify(def)]
  );
}

function literal(v) {
  if (typeof v === 'number') return String(v);
  if (typeof v === 'boolean') return v ? 'true' : 'false';
  return `'${String(v).replace(/'/g, "''")}'`;
}

// Uninstall revokes reach. It never drops the business's rows.
export async function uninstall({ businessId, packageKey }) {
  const row = await one(
    `select * from install where business_id = $1 and package_key = $2`, [busines
  );
  if (!row) return { removed: false };

  await revokeForInstall({ businessId, packageKey });
  await q(`update projection_map set visible = false where install_id = $1`, [row
  await q(`update install set status = 'removed', removed_at = now() where id = $
  await emit({ businessId, topic: 'package.uninstalled', payload: { packageKey }
```

```
  return { removed: true, dataRetained: true };
}
```